```
        payment_details.fetch(:po_num).to_s
      else
        raise "Unknown payment_method #{payment_method}"
      end

      sleep 3 unless Rails.env.test?
      Rails.logger.info "Done Processing Payment"
      OpenStruct.new(succeeded?: true)
    end
  end
```

If you aren't familiar with `OpenStruct`, it's part of Ruby's standard library and provides a quick-and-dirty way to make an object that responds to the methods given to its constructor.[74] In this case, we can call `succeeded?` on the return value from `make_payment`. `OpenStruct` is handy for creating realistic objects from prototype or faked-out code like `Pago`.

With the payment API in hand, you need logic to adapt the payment details that you added in *Defining Additional Fields*, to Pago's API. You'll also move the call to `OrderMailer` into this method, because you don't want to send the email if there was a problem collecting payment.

In a Rails app, when a bit of logic becomes more complex than a line or two of code, you want to move that out of the controller and into a model. You'll create a new method in `Order` called `charge!` that will handle all this logic.

To prepare for this, we first define a `pay_type_params` method in the controller that will capture the parameters to be passed to the model. We put this new method in the bottom of the controller, in the private section:

rails80/depot_qb/app/controllers/orders_controller.rb

```
  def pay_type_params
    if order_params[:pay_type] == "Credit card"
      params.require(:order).permit(:credit_card_number, :expiration_date)
    elsif order_params[:pay_type] == "Check"
      params.require(:order).permit(:routing_number, :account_number)
    elsif order_params[:pay_type] == "Purchase order"
```